

Administrator commands

The 23 verbs an administrator account gets above a programmer, and how to use them.

An administrator account receives everything a programmer account does, plus the 23 verbs below. In OpenQM most of these lived in `SYSTEM` and you reached them by being there; in SD Core they are in the administrator's **own** account VOC, so an SD administrator no longer has to `logto sdsys` to administer anything.

Being an SD administrator means being an elevated Windows administrator. There is no SD password that confers it. Several of these verbs check for elevation and say so rather than failing obscurely — see Security.

Accounts

```
create.account user <name> {administrator | programmer} <ssh|api|both|none>
create.account group <name>
delete.account <name>
modify.account <name> <standard | programmer | administrator | suspended>
modify.account <name> <ssh|api|both|none>
modify.account <name> <sh-on | sh-off | os-on | os-off>
modify.account add <group> <user>
update.accounts {all}
clean.account
```

Covered in full in Account types. The points worth repeating here:

- **One of `ssh`, `api`, `both`, `none` is required** on `create.account user`. There is no default.
- **`modify.account` says what the access is, not what to add.** `modify.account fred api` takes ssh away. The tier keyword works the same way.
- **The tier can be changed after creation**, in either direction, and the VOC is rebuilt at once. `suspended` is a fourth tier that denies entry.
- **`update.accounts` only ever adds VOC records**, never removes them — `modify.account <tier>` is the only thing that removes one.
- **Nothing here changes an administrator's access.** ssh, the API and the operating system are all a rule for that tier; downgrade the account first.

`clean.account` tidies an account's workspace. `update.accounts` is the one you run in each account after upgrading SD.

Grants

```
grant          <account> to   <user>
revoke         <account> from <user>
list.grants    <account>
```

All three need an elevated session.

A grant or a revoke does not take effect until the person signs out of Windows and back in. Entry to an account is Windows group membership, and Windows fixes that at logon. **Somebody you have just revoked keeps the account until they get a new token.** Both verbs print the reminder every time.

Passwords

```
modify.password {<account>}
```

`set.password` **was renamed to** `modify.password` — every account now has a password from the moment it is created, so there is nothing to *set* for the first time.

A password cannot be typed on the command line. `modify.password` refuses one given as an argument. A password on a command line is visible to any local user through Task Manager or `Get-CimInstance Win32_Process`, so SD prompts for it instead, masked.

The password matters for **API logins only**. Console and ssh logins ask for nothing — see Security.

Locks and sessions

```
lock / unlock
list.locks / clear.locks
listu / list.readu
logout
```

`unlock` **is the one to know about.** It clears a record lock left behind by a session that died holding one. Without it the only way to release such a lock is to stop and restart SD, which disconnects everybody.

An earlier changelog entry claimed `unlock` had never worked on any release this port was built from, because its VOC entry carried description text in the type field. **That conclusion was wrong and has been withdrawn.** SD allows a type code followed immediately by comment text — the same rule `PI`, `PI/open` and `UniVerse` follow — so `Verb to unlock records` reads as type `v` with a comment, and `unlock` dispatched normally all along. The record has been rewritten as a bare `v` for consistency, and behaves no differently. `copyyp` carried the same style of entry and was likewise working.

`listu` and `list.readu` report sessions and read locks; `logout` ends another session.

System state

```
config
set.date
```

`config` reports the configuration parameters in force.

`CREATUSR` **is gone.** `config` no longer lists it and `config('CREATUSR')` returns nothing. A `CREATUSR` line in `sd.conf` is still accepted and ignored, so a configuration file brought from a Linux install still works. You can delete the line.

The shell escapes — `sh` and `!`

Who may use `sh` is a list you keep, not a matter of elevation.

This is one of the larger behavioural changes in the port. In earlier builds of it `sh` required an elevated session — and **an ssh session can never be elevated**, so programmers, the people who actually need a shell, were the ones who could never have one.

THE LIST

One record per person, **keyed by the name they sign in with, not by the account name.** The login name follows the person, so it does not change when they `logto` somewhere else.

HOW YOU GRANT IT

Four keywords on `modify.account` do it without your editing anything:

```
modify.account fred sh-on      the sh verb and ! at the prompt
modify.account fred sh-off
```

```
modify.account fred os-on      OS.EXECUTE, and the edit and micro editors
modify.account fred os-off
```

`create.account` takes the two `on` forms. **They are four switches over two fields**, not four names for one state, so `sh-off` leaves `OS.EXECUTE` alone — and the verb prints the resulting record, both fields, every time.

An administrator-tier account gets both fields without being asked, including the account the installer makes for whoever installs SD. That is why an administrator reaches the editors and the shell from an ordinary, unelevated session.

And the four keywords refuse an administrator, in both directions:

```
:modify.account don os-off
don is an administrator and always reaches the operating system
```

An administrator has all three routes — ssh, the API and the operating system — as a rule. Downgrade the account first if that is really what you want.

All three are local only. The grant cannot be taken away and it does not reach past this computer: an administrator signing in from another machine is refused, on ssh and on the API alike, after the password has been checked.

An administrator may not sign in to this machine from another one.

This is not a setting and there is no keyword for it. Administration happens at the console, or through a remote desktop or remote-control product installed as a service. Ordinary and programmer accounts reach the machine remotely as before.

OR EDIT THE RECORD BY HAND

The record is ordinary data. From an **elevated** session, because the file is writable only by an administrator:

```
logto sdsys
ed os.users don
```

The record is two lines and nothing else. To give `don` the shell and the editors:

```
yes
yes
```

In `ed`: `i` to insert, type the two lines, a full stop on its own line to stop inserting, then `fi` to file and exit. **Field 1 is SH**, **field 2 is OS.EX** — the order matters and there is nothing else in the record.

WHAT YOU WANT DON TO HAVE	FIELD 1	FIELD 2
nothing outside SD	<i>no record at all</i>	
the editors, but no shell at the prompt	anything but yes	yes
a shell, but programs may not shell out	yes	anything but yes
both	yes	yes

is the **Windows login name, not the SD account name**. They are usually the same; where they are not, this file wants the one the person signs in with.

The change takes effect on their next command, not at their next login — the list is read when the shell or the editor is asked for.

To take it away, set the field to anything else or delete the record. Deleting the record removes both.

FIELD	CONTROLS	VALUE
1	SH — a shell at the command prompt	yes , or anything else for no
2	OS.EX — OS.EXECUTE from inside a program, and the edit and micro editors	yes , or anything else for no

Field 2 is what lets a programmer use the full-screen editors. They run an editor outside SD, so they are reaching the operating system whatever the VOC tier says. A programmer with the verb and no in field 2 is told the command is not available and what to ask for — see Programmer commands.

A missing record, or a missing file, means no. An installation that has never set up denies both to ordinary accounts.

One case Hand-editing is the only way out of. An account promoted to administrator gets both fields written; an account that was *adopted* over a Windows login which already had a record saying **keeps that record**, and because the four keywords refuse an administrator, no verb can then change it. is the way out. It is rare, and it is the reason this section still documents the file rather than only the keywords.

BOTH FIELDS ARE ENFORCED

used to be unchecked entirely, so **any account that could write a program had the operating system** and the restriction could be walked around by anyone able to type BASIC. Field 2 is now what decides whether may run. The refusal reads *is not permitted to use OS.EXECUTE*.

The two fields are independent, and the useful combination is the third row:

unlisted	sh refused	OS.EXECUTE refused	
SH=yes OS.EX=no	sh RUNS	OS.EXECUTE refused	
SH=no OS.EX=yes	sh refused	OS.EXECUTE RUNS	<- programs may shell out; the person at the prompt may not
elevated	sh runs	OS.EXECUTE RUNS	

SD's own system programs are exempt, so `create.account` and the `sh` verb work as before.

THREE RULES THAT GO WITH IT

Only an administrator can edit the list. `os.users` is read-only to everybody else on disk, and **that ACL is the whole of what stops a user granting themselves a shell. Do not loosen those permissions.**

A listed person gets a real shell — pipes, redirection and chaining all work: `sh dir | more`. Anyone not on the list who reaches `sh` by being elevated still gets the restricted form, which rejects those characters.

An elevated session keeps `sh` whatever the list says, deliberately, so an empty list cannot lock the machine's own administrator out.

NEITHER IS AVAILABLE OVER THE API

`sh` and `OS.EXECUTE` are refused to a session that arrived over the API. **Before this port they were not**, and on Windows that was worse than it sounds: in earlier builds of this port they ran **as the LocalSystem account**, so a remote client could run any command on the machine with full privilege — more than the administrator at the keyboard gets. An API session is no longer treated as an administrator for any purpose.

The full list of the 20

`create.account` · `delete.account` · `modify.account` · `update.accounts` · `clean.account` · `grant` · `revoke` · `list.grants` · `unlock` · `modify.password` · `config` · `listu` · `list.readu` · `list.locks` · `clear.locks` · `lock` · `logout` · `set.date` · `sh` · `!`

Twelve of these left `NEWVOC` on 24 Aug 2026 — `config`, `listu`, `list.readu`, `list.locks`, `clear.locks`, `lock`, `logout`, `set.date`, `sh`, `!`, `clean.account` and `umask`. They are no longer copied into any account's VOC; administrators receive them from `VOC_TEMPLATE` instead. **A programmer or standard account created before that update keeps whichever of them it had**, because `update.accounts` never removes a record.

`umask` **was removed from every tier**. It controls POSIX file-mode bits, which Windows does not use for security — all file access runs through the ACLs an SD account already receives. Setting a umask on Windows produces no security effect and would only be misleading. The record types stay compiled, but no VOC in any tier dispatches to them.

